

Documentación del proyecto

Se debe realizar la siguiente documentación por cada proyecto.

Fase 1. Contextualización

- **¿Qué se debe hacer?**

Desarrollar una aplicación con interfaz gráfica que permita determinar los colores de las bandas de una resistencia a partir de su valor en ohmios. Además, el programa debe generar un número n de resistencias aleatorias y calcular la suma total de sus valores si estuvieran conectadas en serie y en paralelo.

- **¿Qué se necesita?**

Un entorno de desarrollo para Python (como VS Code), la biblioteca tkinter para la interfaz gráfica, y la implementación de una estructura de datos basada en listas enlazadas (clases NodoLista y Lista) para almacenar y manipular dinámicamente los valores aleatorios de las resistencias sin usar las listas nativas de Python.

- **¿Qué elementos se identifican?**

El valor numérico de la resistencia (capturado desde la interfaz), los colores correspondientes a las bandas (primer dígito, segundo dígito y multiplicador), la cantidad n de resistencias a generar proporcionada por el usuario, los valores aleatorios de cada resistencia (entre 10 y 1,000,000,000), y las ecuaciones matemáticas para la suma de circuitos en serie y paralelo.

Fase 2. Definición de los requerimientos funcionales

Se debe elaborar el listado de los requerimientos del contexto planteado.

Requerimientos

Para la documentación de cada requerimiento funcional utilizar el siguiente esquema descriptivo.

Nombre	R1-Determinación de colores de resistencia
Descripción	El sistema debe calcular y mostrar en la interfaz gráfica los colores correspondientes a la primera banda, segunda banda y multiplicador de una resistencia en función del valor en ohmios ingresado por el usuario.
Entrada	Un número entero que representa el valor de la resistencia en ohmios.
Salidas	Una representación visual y en texto de los colores asociados a las bandas de la resistencia.

Nombre	R2-Cálculo de resistencias en serie y paralelo
---------------	--

Descripción	El sistema debe generar n valores aleatorios enteros dentro del rango de 10 a 1,000,000,000, almacenar estos valores instanciando la clase Lista (lista enlazada), e iterar sobre los nodos para calcular la resistencia total equivalente aplicando las fórmulas de suma en serie y paralelo.
Entrada	Un número entero n que indica la cantidad total de resistencias a generar.
Salidas	El valor total resultante, expresado en ohmios, para la configuración en serie y la configuración en paralelo.

Fase 3. Diseño

Determinar las clases que conforman el sistema, junto con los atributos y los métodos y crear el diagrama de clases UML, así como sus relaciones de herencia (si aplica).

Nombre de la clase: NodoLista
Atributos datos siguienteNodo
Métodos __init__() obtenerObject() obtenerSiguiente()

Nombre de la clase: Lista
Atributos nombre primerNodo ultimoNodo
Métodos __init__() insertarAlFrente() insertarAlFinal() estaVacia() calcularResistenciaSerie() calcularResistenciaParalelo()

Nombre de la clase: InterfazResistencias
Atributos ventana input_valor_ohm input_cantidad_n label_resultados canvas_dibujo

Métodos __init__() calcular_colores() generar_y_sumar_resistencias() actualizar_interfaz()
--

Fase 4. Codificación

Se deben escribir los programas fuentes necesarios, basados en el diagrama de clase de la fase anterior:

- Implementación de los métodos (construcción de la parte lógica):
Se adaptó la clase Lista para incluir métodos de recorrido que visitan cada NodoLista sumando los datos contenidos.
- Codificación de fórmulas:
Para la suma en serie, se inicializa una variable en 0 y se aplica un ciclo while que avanza nodo por nodo haciendo la asignación $Rts = Rts + \text{actual.datos}$.
Para la suma en paralelo, se aplica la fórmula acumulando el inverso de los valores $Rtp_acumulador = Rtp_acumulador + (1 / \text{actual.datos})$ y finalmente se obtiene el inverso del resultado total $Rtp = 1 / Rtp_acumulador$.
- Asignaciones y Condicionales:
Se emplean estructuras condicionales (if/elif/else) para validar que las entradas de texto en la interfaz gráfica sean números enteros válidos y mayores a cero. Para la lógica de colores, se extrae el primer carácter (primer dígito), el segundo carácter (segundo dígito) y se cuenta la longitud restante de la cadena para determinar el multiplicador, asignando los colores mediante un índice.
- Documentación del código.

Fase 5. Pruebas

Aquí se debe:

- Verificar que el programa cumpla con los requerimientos establecidos.
- Construcción de los documentos de entrega final.
- Adjuntar evidencias de resultados de la ejecución (impresión de pantallas).

Resultados obtenidos (Pruebas de Escritorio):

- Se verificó el programa cumpliendo con los requerimientos establecidos, ingresando valores de prueba documentados. Al ingresar el valor "160", el sistema determina exitosamente los colores Marrón, Azul y Marrón. Al ingresar el valor "5800", el sistema procesa Verde, Gris y Rojo.
- Se probó la instanciación correcta de la lista enlazada solicitando $n=50$ resistencias, confirmando que los cálculos en serie y paralelo no arrojan errores de división por cero y muestran el resultado esperado en ohmios.

CAPTURAS DEL FUNCIONAMIENTO DEL PROGRAMA

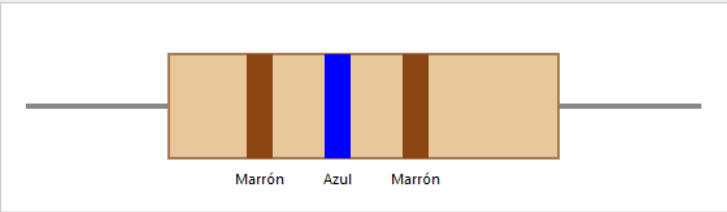
Proyecto de Arreglos - Resistencia Eléctrica

Colores de Resistencia Serie y Paralelo

Ingresa el valor de la resistencia (Ω):

160

Calcular colores



Marrón Azul Marrón

Primera banda: Marrón
Segunda banda: Azul
Multiplicador: Marrón

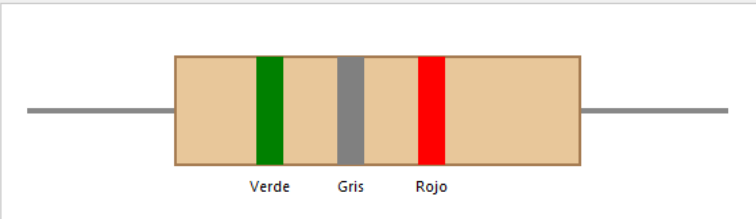
Proyecto de Arreglos - Resistencia Eléctrica

Colores de Resistencia Serie y Paralelo

Ingresa el valor de la resistencia (Ω):

5800

Calcular colores



Verde Gris Rojo

Primera banda: Verde
Segunda banda: Gris
Multiplicador: Rojo

Proyecto de Arreglos - Resistencia Eléctrica

Colores de Resistencia Serie y Paralelo

Cantidad de resistencias a generar (n):

50

Generar y calcular

Resistencias generadas (nodos de la lista enlazada):

R1 = 178512340 Ω
R2 = 292774834 Ω
R3 = 538874673 Ω
R4 = 167934621 Ω
R5 = 776683800 Ω
R6 = 837933257 Ω
R7 = 790914909 Ω
R8 = 853338369 Ω

Rts (Serie) = 22,996,695,401 Ω
Rtp (Paralelo) = 4,930,952.0064 Ω